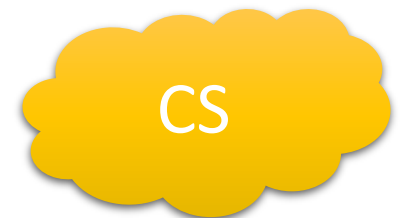
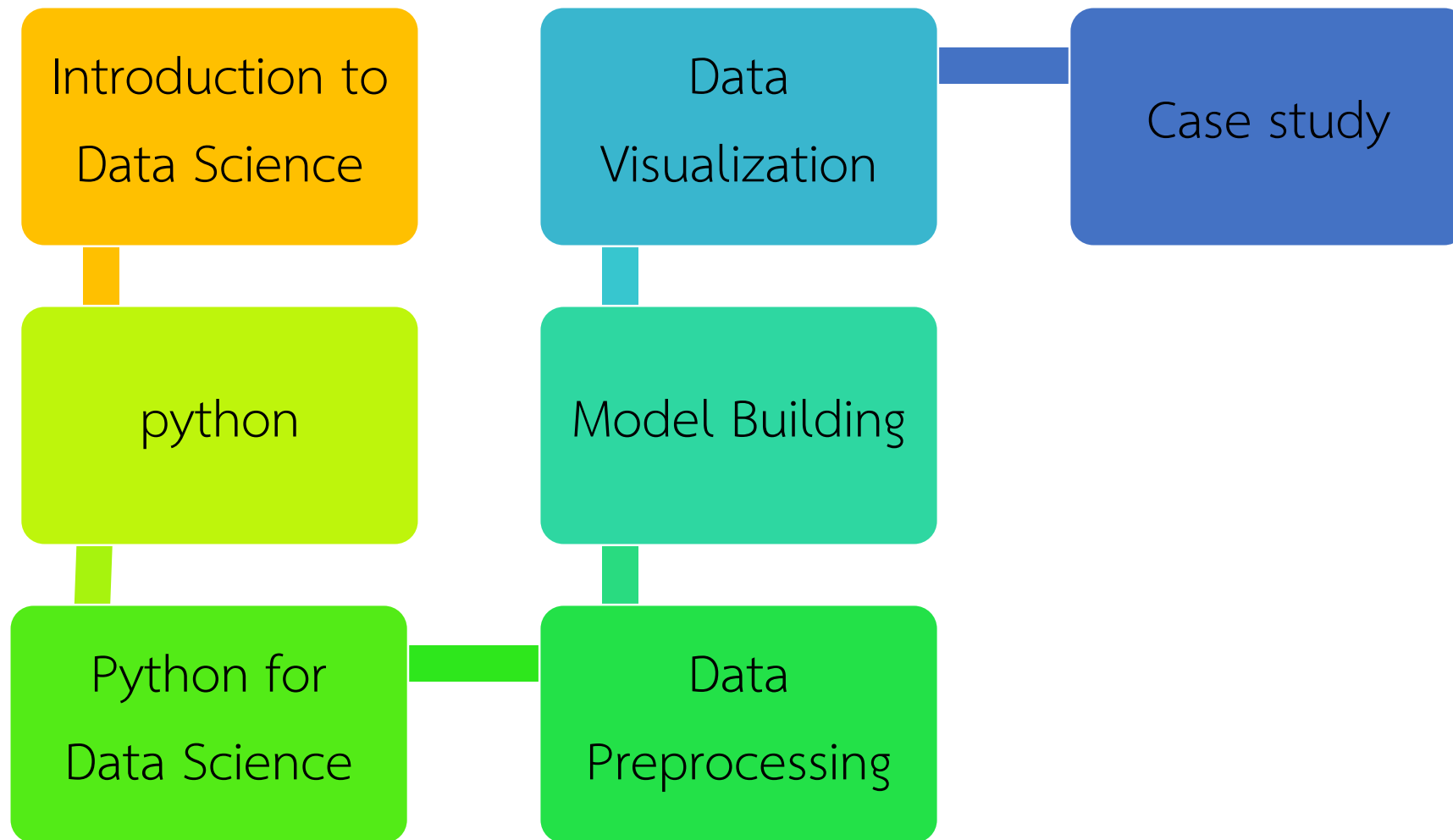


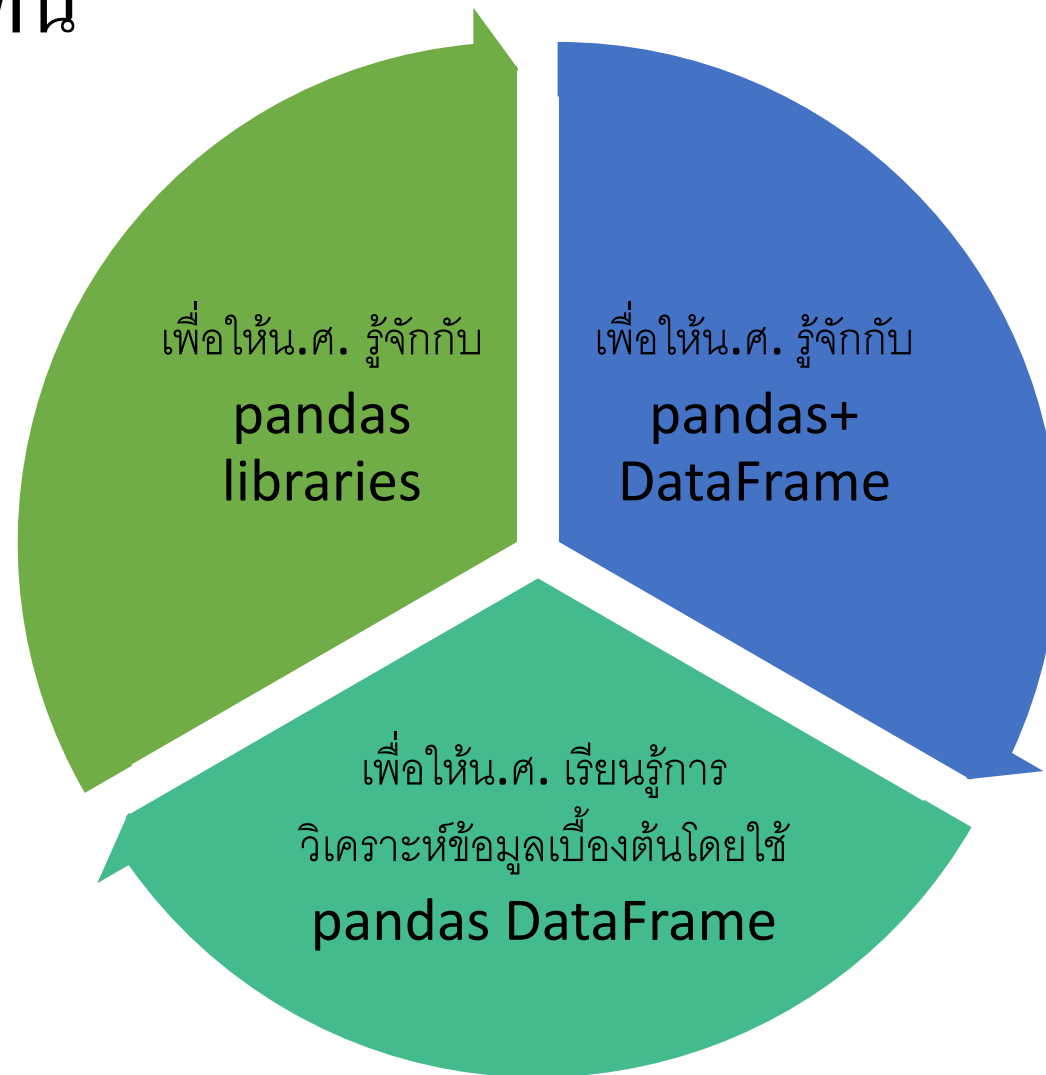
รหัสวิชา 2-233-225
ชื่อวิชา วิทยาศาสตร์ข้อมูลเบื้องต้น
(Introduction to Data Science)
สัปดาห์ที่ 10

CS

เนื้อหาของรายวิชา

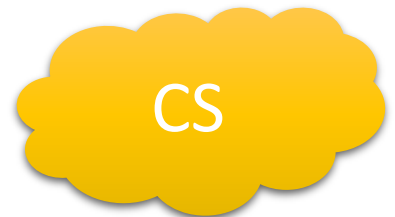
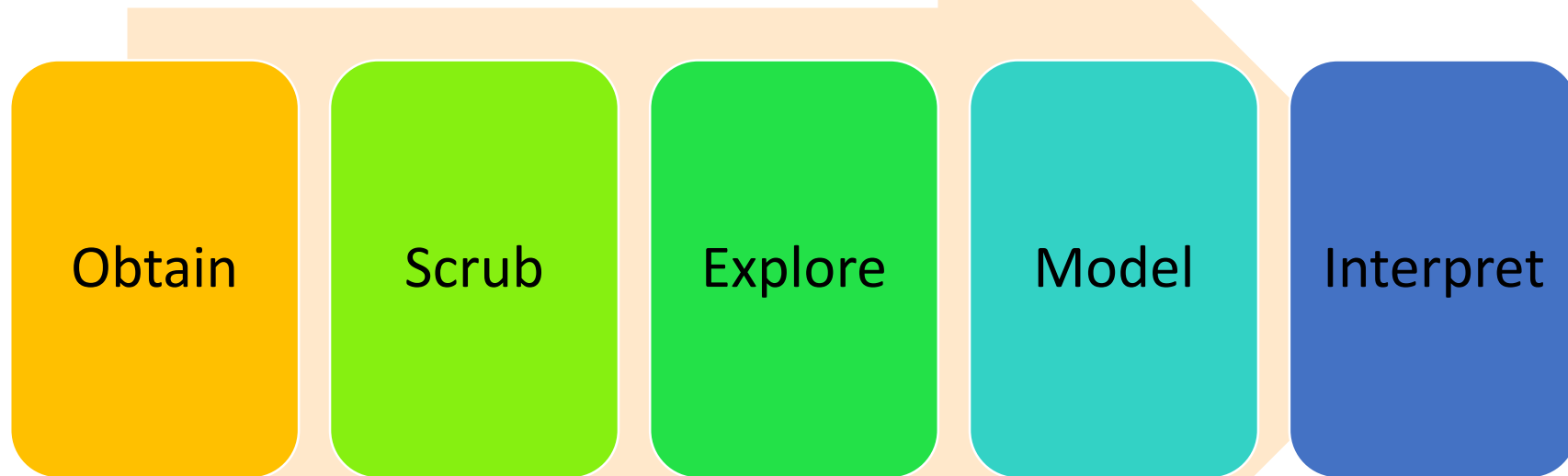


วัตถุประสงค์ของบทนี้

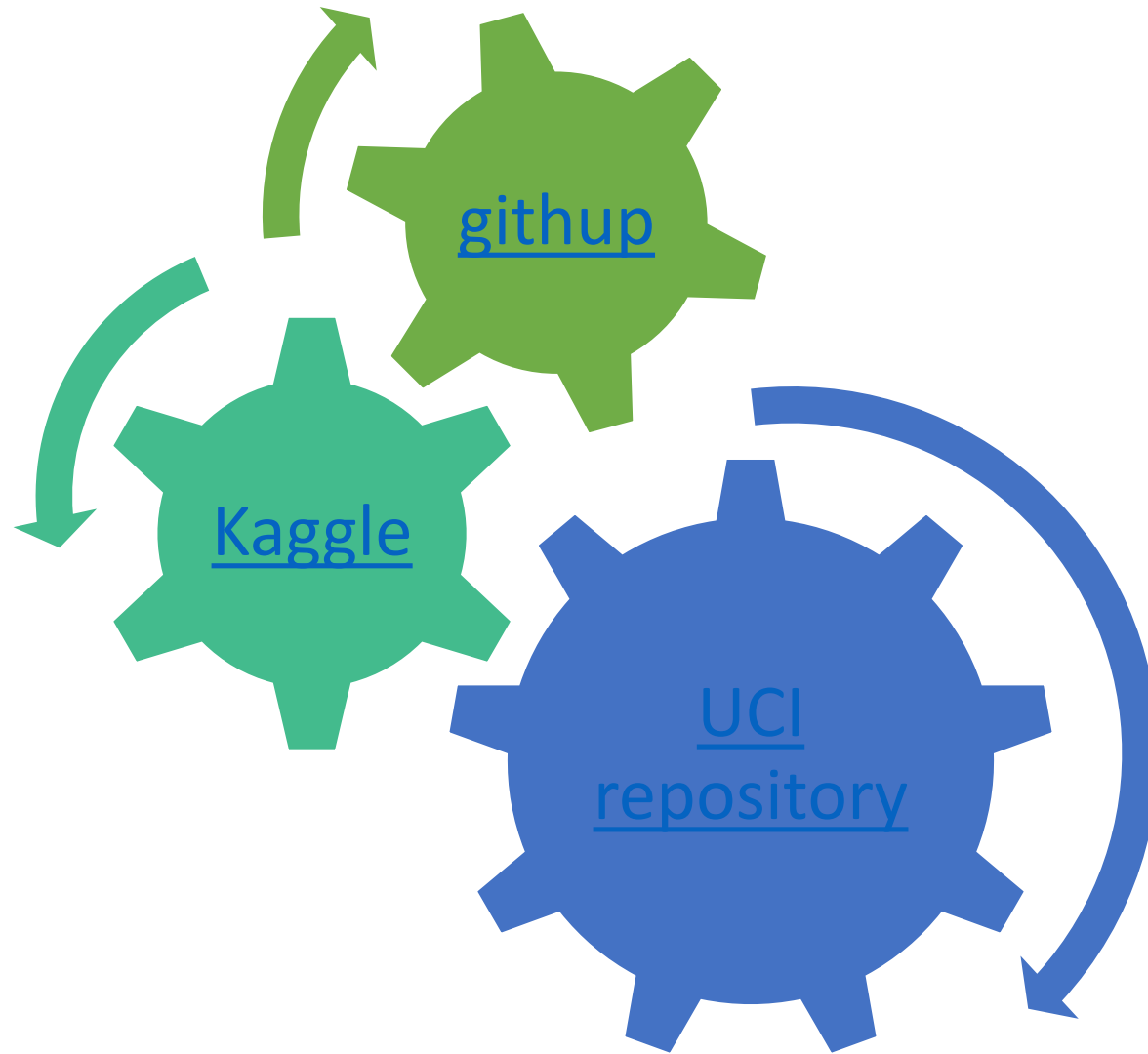


ขั้นตอนการทำ Data Science

Data Science Pipeline



Data Source



<https://www.kaggle.com/>

<https://archive.ics.uci.edu/ml/index.php>

<https://github.com/awesomedata/awesome-public-datasets>

Pandas

“Pandas is actually a powerful library that enables Python to work with structured datasets using panel data or dataframes. Pandas started life as a project by Wes McKinney in 2008 with the aim of enabling Python to be a more practical statistical computing environment.”

[Jesús Rogel-Salazar, 2017]

Key Features of Pandas

Fast and efficient

Tools for loading data

handling of missing data

Reshaping and pivoting

Label-based slicing, indexing and subsetting

deleted or inserted data

Group by data for aggregation and transformations

merging and joining of data

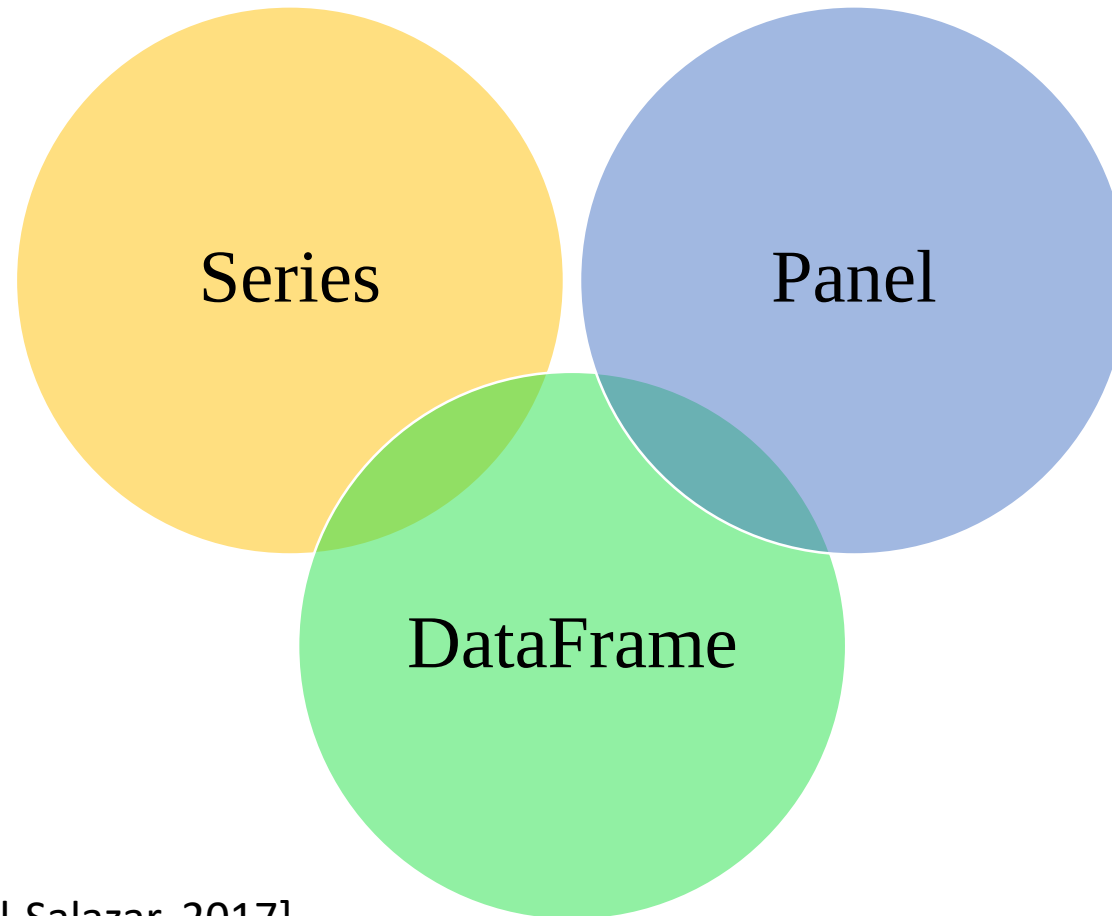
Time Series functionality

Getting Started with **pandas**

```
import pandas as pd
```

[Jesús Rogel-Salazar, 2017]

data structures



[Jesús Rogel-Salazar, 2017]

วิเคราะห์ข้อมูลเบื้องต้นเชิงพรรณนา

```
In [1]: import pandas as pd
```

```
In [2]: df = pd.read_csv('https://gist.githubusercontent.com/omarish/5687264/raw/7e5c814ce6ef33e25d5259c1fe79463c190800d9/mpg.csv')
```

```
In [4]: df.head()
```

```
Out[4]:
```

	mpg	cylinders	displacement	horsepower	weight	acceleration	model_year	origin	name
0	18.0	8	307.0	130	3504	12.0	70	1	chevrolet chevelle malibu
1	15.0	8	350.0	165	3693	11.5	70	1	buick skylark 320
2	18.0	8	318.0	150	3436	11.0	70	1	plymouth satellite
3	16.0	8	304.0	150	3433	12.0	70	1	amc rebel sst
4	17.0	8	302.0	140	3449	10.5	70	1	ford torino

```
In [5]: df.tail()
```

```
Out[5]:
```

	mpg	cylinders	displacement	horsepower	weight	acceleration	model_year	origin	name
393	27.0	4	140.0	86	2790	15.6	82	1	ford mustang gl
394	44.0	4	97.0	52	2130	24.6	82	2	vw pickup
395	32.0	4	135.0	84	2295	11.6	82	1	dodge rampage
396	28.0	4	120.0	79	2625	18.6	82	1	ford ranger
397	31.0	4	119.0	82	2720	19.4	82	1	chevy s-10

```
In [6]: df.shape
```

```
Out[6]: (398, 9)
```

<https://gist.github.com/omarish/5687264/>
<https://gist.github.com/omarish/5687264/raw/7e5c814ce6ef33e25d5259c1fe79463c190800d9/mpg.csv>

pandas-count frequency of string

```
In [8]: df.name.value_counts()
Out[8]: ford pinto          6
      amc matador          5
      ford maverick        5
      toyota corolla        5
      toyota corona        4
      amc gremlin          4
      amc hornet           4
      chevrolet impala     4
      peugeot 504          4
      chevrolet chevette   4
      ford gran torino     3
      chevrolet citation   3
      plymouth duster      3
      dodge colt           3
      volkswagen dasher    3
      chevrolet caprice classic 3
      pontiac catalina     3
      plymouth fury iii    3
      ford galaxie 500     3
      honda civic          3
      chevrolet vega       3
      chevrolet nova       3
      toyota mark ii       2
      amc concord          2
      amc matador (sw)     2
      chevrolet malibu     2
      ford ltd             2
      datsun 710           2
      oldsmobile cutlass salon brougham 2
      opel manta           2
      ..
      oldsmobile cutlass ciera (diesel) 1
      amc spirit dl         1
      ford f100            1
      mercurv capri 2000   1
```

<https://www.youtube.com/watch?v=ciXCePAEvxE>

correlation

In [9]: `df.corr()`

Out[9]:

	mpg	cylinders	displacement	weight	acceleration	model_year	origin
mpg	1.000000	-0.775398	-0.804203	-0.831741	0.420289	0.579267	0.563450
cylinders	-0.775398	1.000000	0.950721	0.896017	-0.505419	-0.348746	-0.562543
displacement	-0.804203	0.950721	1.000000	0.932824	-0.543684	-0.370164	-0.609409
weight	-0.831741	0.896017	0.932824	1.000000	-0.417457	-0.306564	-0.581024
acceleration	0.420289	-0.505419	-0.543684	-0.417457	1.000000	0.288137	0.205873
model_year	0.579267	-0.348746	-0.370164	-0.306564	0.288137	1.000000	0.180662
origin	0.563450	-0.562543	-0.609409	-0.581024	0.205873	0.180662	1.000000

<https://www.youtube.com/watch?v=ciXCePAEvxE>

การแสดงแถวข้อมูลใน pandas โดยใช้ head, tail และ sample

```
In [14]: import pandas as pd  
%matplotlib inline
```

```
In [15]: pd.__version__
```

```
Out[15]: '0.22.0'
```

```
In [20]: df = pd.read_csv('data/province.csv')
```

```
In [21]: df
```

```
Out[21]:
```

	Sr.No	Province	Capital	Population	Area (km ²)	Population Density	Largest Metropolitan Area	Abbr.	ISO	FIPS
0	1	Bangkok	Bangkok	5,701,394	1,568.70	3,634.50	Bangkok Metropolitan Area	BKK	TH-10	TH40
1	2	Amnat Charoen	Amnat Charoen	372,137	3,161.20	117.7	Amnat Charoen	ACR	TH-37	TH77
2	3	Ang Thong	Ang Thong	284,970	968.4	294.3	Ang Thong	ATG	TH-15	TH35
3	4	Bueng Kan	Bueng Kan	385,053	4,305	89.4	Bueng Kan	ATG	TH-38	TH35
4	5	Buriram	Buriram	1,553,765	10,322.90	150.5	Buriram	BRM	TH-31	TH28
5	6	Chachoengsao	Chachoengsao	673,933	5,351.00	125.9	Chachoengsao	CCO	TH-24	TH44
6	7	Chainat	Chainat	334,934	2,469.70	135.6	Chainat	CNT	TH-18	TH32
7	8	Chaiyaphum	Chaiyaphum	1,127,423	12,778.30	88.2	Chaiyaphum	CPM	TH-36	TH26
8	9	Chanthaburi	Chanthaburi	514,616	6,338.00	81.2	Chanthaburi	CTI	TH-22	TH48
9	10	Chiang Mai	Chiang Mai	1,640,479	20,107.00	81.6	Chiang Mai	CMI	TH-50	TH02
10	11	Chiang Rai	Chiang Rai	1,198,218	11,678.40	102.6	Chiang Rai	CRI	TH-57	TH03
11	12	Chonburi	Chonburi	1,316,293	4,363.00	301.7	Pattaya	CBI	TH-20	TH46
12	13	Chumphon	Chumphon	489,964	6,009.00	81.5	Chumphon	CPN	TH-86	TH58
13	14	Kalasin	Kalasin	982,578	6,946.70	141.4	Kalasin	KSN	TH-46	TH23
14	15	Kamphaeng Phet	Kamphaeng Phet	727,093	8,607.50	84.5	Kamphaeng Phet	KPT	TH-62	TH11
15	16	Kanchanaburi	Kanchanaburi	839,776	19,483.20	43.1	Kanchanaburi	KRI	TH-71	TH50
16	17	Khon Kaen	Khon Kaen	1,767,601	10,886.00	162.4	Khon Kaen	KKN	TH-40	TH22
17	18	Krabi	Krabi	432,704	4,708.50	91.9	Krabi	KRI	TH-81	TH63

การแสดงแถวข้อมูลใน pandas โดยใช้ **head**, **tail** และ **sample**

```
In [22]: df.head()
```

```
Out[22]:
```

	Sr.No	Province	Capital	Population	Area (km ²)	Population Density	Largest Metropolitan Area	Abbr.	ISO	FIPS
0	1	Bangkok	Bangkok	5,701,394	1,568.70	3,634.50	Bangkok Metropolitan Area	BKK	TH-10	TH40
1	2	Amnat Charoen	Amnat Charoen	372,137	3,161.20	117.7	Amnat Charoen	ACR	TH-37	TH77
2	3	Ang Thong	Ang Thong	284,970	968.4	294.3	Ang Thong	ATG	TH-15	TH35
3	4	Buang Kan	Buang Kan	385,053	4,305	89.4	Buang Kan	ATG	TH-38	TH35
4	5	Buriram	Buriram	1,553,765	10,322.90	150.5	Buriram	BRM	TH-31	TH28

<https://www.youtube.com/watch?v=ciXCePAEvxE>

การแสดงแถวข้อมูลใน pandas โดยใช้ head, **tail** และ sample

```
In [23]: df.tail()
```

```
Out[23]:
```

	Sr.No	Province	Capital	Population	Area (km ²)	Population Density	Largest Metropolitan Area	Abbr.	ISO	FIPS
72	73	Udon Thani	Udon Thani	1,544,788	11,730.30	131.7	Udon Thani	UDN	TH-41	TH78
73	74	Uthai Thani	Uthai Thani	327,959	6,730.30	48.7	Uthai Thani	UTI	TH-61	TH15
74	75	Uttaradit	Uttaradit	462,618	7,838.60	59	Uttaradit	UTD	TH-53	TH10
75	76	Yala	Yala	487,380	4,521.10	107.8	Yala	YLA	TH-95	TH70
76	77	Yasothon	Yasothon	539,257	4,161.70	129.6	Yasothon	YST	TH-35	TH72

<https://www.youtube.com/watch?v=ciXCePAEvxE>

การแสดงแถวข้อมูลใน pandas โดยใช้ head, tail และ sample

```
In [25]: df.sample()
```

Out[25]:

	Sr.No	Province	Capital	Population	Area (km ²)	Population Density	Largest Metropolitan Area	Abbr.	ISO	FIPS
71	72	Ubon Ratchathani	Ubon Ratchathani	1,813,088	15,744.80	115.2	Ubon Ratchathani	UBN	TH-34	TH75

```
In [26]: df.sample(n=5)
```

Out[26]:

	Sr.No	Province	Capital	Population	Area (km ²)	Population Density	Largest Metropolitan Area	Abbr.	ISO	FIPS
8	9	Chanthaburi	Chanthaburi	514,616	6,338.00	81.2	Chanthaburi	CTI	TH-22	TH48
66	67	Surat Thani	Surat Thani	1,000,383	12,891.50	77.6	Surat Thani	SNI	TH-84	TH80
63	64	Songkhla	Songkhla	1,357,023	7,393.90	183.5	Hat Yai	SKA	TH-90	TH88
12	13	Chumphon	Chumphon	489,964	6,009.00	81.5	Chumphon	CPN	TH-86	TH58
27	28	Nakhon Phanom	Nakhon Phanom	703,392	5,512.70	127.6	Nakhon Phanom	NPM	TH-48	TH73

<https://www.youtube.com/watch?v=ciXCePAEvxE>

การแสดงแถวข้อมูลใน pandas โดยใช้ head, tail และ sample

```
In [27]: df.sample(frac=.1)
```

```
Out[27]:
```

	Sr.No	Province	Capital	Population	Area (km²)	Population Density	Largest Metropolitan Area	Abbr.	ISO	FIPS
55	56	Sakon Nakhon	Sakon Nakhon	1,122,905	9,605.80	116.9	Sakon Nakhon	SNK	TH-47	TH20
73	74	Uthai Thani	Uthai Thani	327,959	6,730.30	48.7	Uthai Thani	UTI	TH-81	TH15
9	10	Chiang Mai	Chiang Mai	1,640,479	20,107.00	81.6	Chiang Mai	CMI	TH-50	TH02
67	68	Surin	Surin	1,381,761	8,124.10	170.1	Surin	SRN	TH-32	TH29
43	44	Phichit	Phichit	552,690	4,531.00	122	Phichit	PCT	TH-66	TH13
56	57	Samut Prakan	Samut Prakan	1,185,180	1,004.10	1,180.30	Bangkok Metropolitan Area	SPK	TH-11	TH42
38	39	Phang Nga	Phang Nga	253,112	4,170.00	60.7	Thai Mueang	PNA	TH-82	TH61
34	35	Nong Khai	Nong Khai City	912,937	3,027.00	124.5	Nong Khai City	NKI	TH-43	TH17

```
In [28]: df.Province.sample(n=5)
```

```
Out[28]: 5    Chachoengsao
16    Khon Kaen
47    Phuket
8     Chanthaburi
37    Pattani
Name: Province, dtype: object
```

<https://www.youtube.com/watch?v=ciXCePAEvxE>

ประเภทข้อมูล (data types) ใน pandas

```
In [1]: import pandas as pd
```

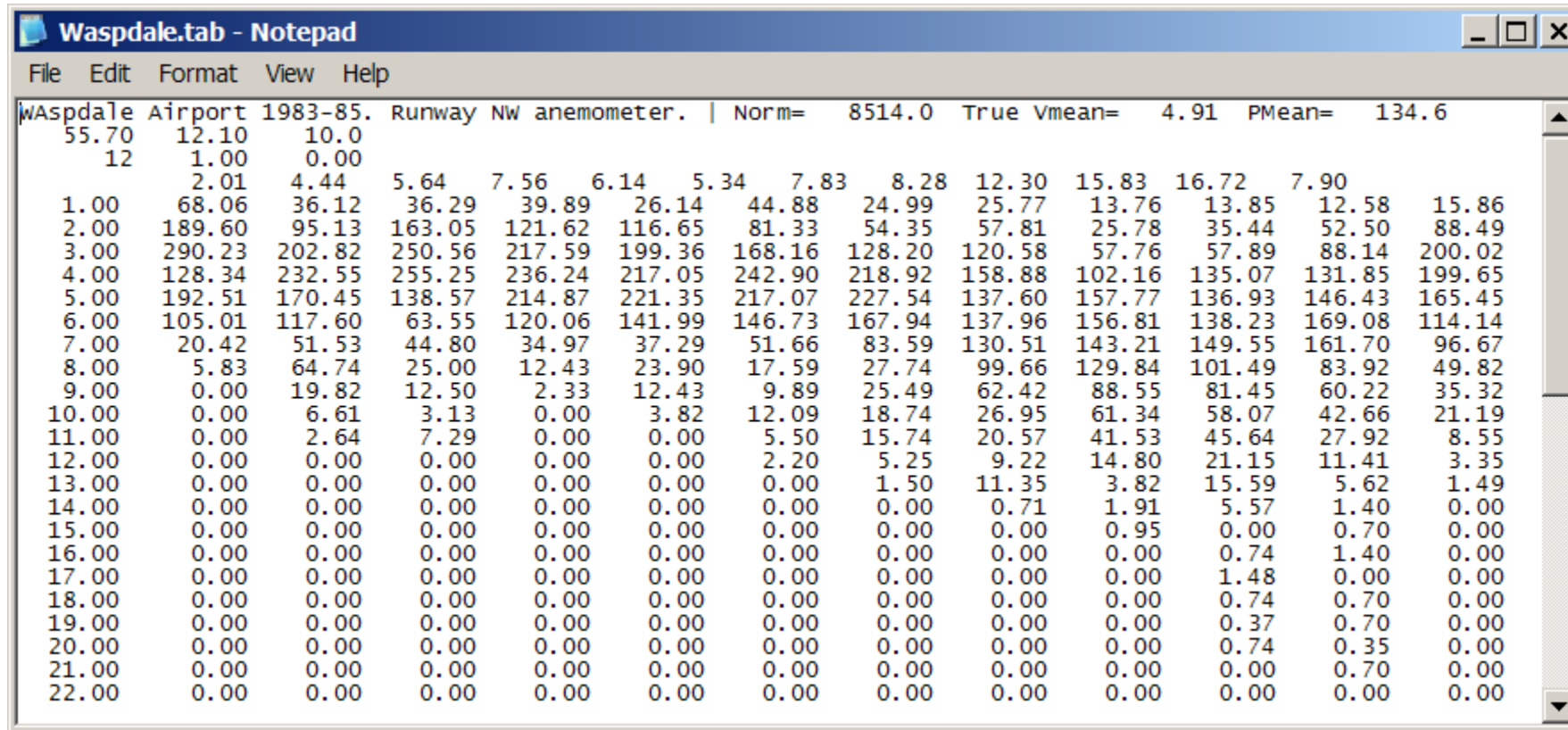
```
In [3]: df = pd.read_table('https://github.com/prasertcbs/tutorial/raw/master/employee.tsv')  
df.head()
```

Out[3]:

	gender	dob	occupation	work_exp	salary	own_house	children
0	M	31-Jul-89	nurse	2.2	223700.0	True	1
1	NaN	30-Oct-92	dentist	6.7	121100.0	True	1
2	F	NaN	accountant	9.0	NaN	False	2
3	F	17-Sep-87	engineer	1.7	79900.0	False	0
4	M	24-Jan-93	teacher	3.6	32800.0	False	2

<https://github.com/prasertcbs/tutorial/blob/master/employee.tsv>

Example: TSV file



Waspdale Airport 1983-85. Runway NW anemometer. Norm= 8514.0 True Vmean= 4.91 PMean= 134.6														
55.70	12.10	10.0												
12	1.00	0.00												
	2.01	4.44	5.64	7.56	6.14	5.34	7.83	8.28	12.30	15.83	16.72	7.90		
1.00	68.06	36.12	36.29	39.89	26.14	44.88	24.99	25.77	13.76	13.85	12.58	15.86		
2.00	189.60	95.13	163.05	121.62	116.65	81.33	54.35	57.81	25.78	35.44	52.50	88.49		
3.00	290.23	202.82	250.56	217.59	199.36	168.16	128.20	120.58	57.76	57.89	88.14	200.02		
4.00	128.34	232.55	255.25	236.24	217.05	242.90	218.92	158.88	102.16	135.07	131.85	199.65		
5.00	192.51	170.45	138.57	214.87	221.35	217.07	227.54	137.60	157.77	136.93	146.43	165.45		
6.00	105.01	117.60	63.55	120.06	141.99	146.73	167.94	137.96	156.81	138.23	169.08	114.14		
7.00	20.42	51.53	44.80	34.97	37.29	51.66	83.59	130.51	143.21	149.55	161.70	96.67		
8.00	5.83	64.74	25.00	12.43	23.90	17.59	27.74	99.66	129.84	101.49	83.92	49.82		
9.00	0.00	19.82	12.50	2.33	12.43	9.89	25.49	62.42	88.55	81.45	60.22	35.32		
10.00	0.00	6.61	3.13	0.00	3.82	12.09	18.74	26.95	61.34	58.07	42.66	21.19		
11.00	0.00	2.64	7.29	0.00	0.00	5.50	15.74	20.57	41.53	45.64	27.92	8.55		
12.00	0.00	0.00	0.00	0.00	0.00	2.20	5.25	9.22	14.80	21.15	11.41	3.35		
13.00	0.00	0.00	0.00	0.00	0.00	0.00	1.50	11.35	3.82	15.59	5.62	1.49		
14.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.71	1.91	5.57	1.40	0.00		
15.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.95	0.00	0.70	0.00		
16.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.74	1.40	0.00		
17.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	1.48	0.00	0.00		
18.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.74	0.70	0.00		
19.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.37	0.70	0.00		
20.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.74	0.35	0.00		
21.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.70	0.00		
22.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00		

[Jake VanderPlas, 2017]

ประเภทข้อมูล (data types) ใน pandas

In [4]: `df.info()`

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 10 entries, 0 to 9
Data columns (total 7 columns):
gender      9 non-null object
dob         8 non-null object
occupation  10 non-null object
work_exp    10 non-null float64
salary      8 non-null float64
own_house   10 non-null bool
children    10 non-null int64
dtypes: bool(1), float64(2), int64(1), object(3)
memory usage: 570.0+ bytes
```

<https://www.youtube.com/watch?v=ciXCePAEvxE>

ประเภทข้อมูล (data types) ใน pandas –convert date

```
In [5]: df = pd.read_table('https://github.com/prasertcbs/tutorial/raw/master/employee.tsv', parse_dates = ['dob'])
df.head()
```

Out[5]:

	gender	dob	occupation	work_exp	salary	own_house	children
0	M	1989-07-31	nurse	2.2	223700.0	True	1
1	NaN	1992-10-30	dentist	6.7	121100.0	True	1
2	F	NaT	accountant	9.0	NaN	False	2
3	F	1987-09-17	engineer	1.7	79900.0	False	0
4	M	1993-01-24	teacher	3.6	32800.0	False	2

<https://github.com/prasertcbs/tutorial/blob/master/employee.tsv>

<https://www.youtube.com/watch?v=ciXCePAEvxE>

ประเภทข้อมูล (data types) ใน pandas

```
In [6]: df.salary.mean()
```

```
Out[6]: 122612.5
```

```
In [7]: df.salary.sum()
```

```
Out[7]: 980900.0
```

```
In [8]: df.occupation.value_counts()
```

```
Out[8]: teacher      2  
nurse      2  
accountant  1  
lawyer      1  
dentist     1  
engineer    1  
developer    1  
police      1  
Name: occupation, dtype: int64
```

<https://www.youtube.com/watch?v=ciXCePAEvxE>

ประเภทข้อมูล (data types) ใน pandas

```
In [9]: df.occupation.str.upper()
```

```
Out[9]: 0      NURSE  
1     DENTIST  
2  ACCOUNTANT  
3    ENGINEER  
4     TEACHER  
5    DEVELOPER  
6      NURSE  
7     LAWYER  
8     TEACHER  
9     POLICE  
Name: occupation, dtype: object
```

<https://www.youtube.com/watch?v=ciXCePAEvxE>

ประเภทข้อมูล (data types) ใน pandas

```
In [10]: pd.to_datetime('today')
```

```
Out[10]: Timestamp('2018-08-12 00:00:00')
```

```
In [11]: pd.to_datetime('today').year
```

```
Out[11]: 2018
```

<https://www.youtube.com/watch?v=ciXCePAEvxE>

ประเภทข้อมูล (data types) ใน pandas

```
In [16]: df.own_house.value_counts()
```

```
Out[16]: True      5  
        False     5  
        Name: own_house, dtype: int64
```

```
In [17]: df.describe()
```

```
Out[17]:
```

	work_exp	salary	children	age
count	10.000000	8.000000	10.000000	8.000000
mean	4.450000	122612.500000	0.900000	27.125000
std	2.429106	86579.978509	0.737865	2.900123
min	1.700000	32800.000000	0.000000	24.000000
25%	2.475000	61400.000000	0.250000	24.750000
50%	3.850000	100500.000000	1.000000	26.500000
75%	6.275000	161900.000000	1.000000	29.500000
max	9.000000	272500.000000	2.000000	31.000000

<https://www.youtube.com/watch?v=ciXCePAEvxE>

Missing value

ว/ด/ป	รายการ	ราคา	จำนวน	ราคารวม
02/11//2018	ขนมปัง	20	2	40
02/11//2018	นม	10	5	50
03/11//2018	นม	10	2	NAN
03/11//2018	น้ำอัดลม	15	NAN	30
04/11//2018	น้ำอัดลม	NAN	1	15
05/11//2018	น้ำอัดลม	15	2	30
05/11//2018	นม	NAN	2	20
06/11//2018	ขนมปัง	NAN	2	NAN
07/11//2018	นม	10	NAN	30
07/11//2018	ขนมปัง	20	5	100

ตัวอย่างที่ 1

Ignore the tuple

ว/ด/ป	รายการ	ราคา	จำนวน	ราคารวม
02/11//2018	ขนมปัง	20	2	40
02/11//2018	นม	10	5	50
03/11//2018	นม	10	2	
03/11//2018	น้ำอัดลม	15		30
04/11//2018	น้ำอัดลม		1	15
05/11//2018	น้ำอัดลม	15	2	30
05/11//2018	นม		2	20
06/11//2018	ขนมปัง		2	
07/11//2018	นม	10		30
07/11//2018	ขนมปัง	20	5	100

ตัวอย่างที่ 2

Add “unknown”

รหัสนักศึกษา	ชั้นปี	ที่อยู่	SA	แคล 1	เขียนโปรแกรม 1	เกรดเฉลี่ย
5043225-8	2	123 Bangkok Thailand	A	A	A	4.00
5043222-1	2	5 Bangkok Thailand	unknown	B	C+	unknown
5043224-5	2	5 Bangkok Thailand	A	unknown	unknown	2.5
5043225-7	2	5 Bangkok Thailand	B+	C+	unknown	unknown
5043225-2	2	45 Bangkok Thailand	D	C	B	2.5

ตัวอย่างที่ 3

attribute mean

emplID	sex	Age	position	department	salary
604	M	26	ผู้จัดการ	HR	13,000
607	F	50	ผู้ช่วยผู้จัดการ	FIN	40,000
486	F	38	ผู้ช่วยผู้จัดการ	FIN	25,000
229	M	32	พนักงานระดับต้น	MKT	45,850
901	F	55	รองผู้จัดการ	MKT	82,500
944	F	35	พนักงานระดับต้น	HR	38,000
405	M	42	พนักงานระดับต้น	FIN	35,000
665	F	42	ผู้ชำนาญงาน	ACC	21,000
41	F	52	ผู้จัดการ	MKT	45,850
171	F	29	พนักงานระดับต้น	HR	25,000
51	F	58	ผู้จัดการ	MKT	101,500
724	M	45	ผู้ช่วยผู้จัดการ	MKT	77,500

ตัวอย่างที่ 4

attribute mean of the same label

emplID	sex	Age	position	department	salary
604	M	26	ผู้จัดการ	HR	13,000
607	F	50	ผู้ช่วยผู้จัดการ	FIN	40,000
486	F	38	ผู้ช่วยผู้จัดการ	FIN	25,000
229	M	32	พนักงานระดับต้น	MKT	32,667
901	F	55	รองผู้จัดการ	MKT	82,500
944	F	35	พนักงานระดับต้น	HR	38,000
405	M	32	พนักงานระดับต้น	FIN	35,000
665	F	30	ผู้ชำนาญงาน	ACC	21,000
41	F	52	ผู้จัดการ	MKT	36,000
171	F	29	พนักงานระดับต้น	HR	25,000
51	F	58	ผู้จัดการ	MKT	101,500
724	M	45	ผู้ช่วยผู้จัดการ	MKT	77,500

Handling Missing Values



dropping

- `dataFrame.dropna()`



filling

- `dataFrame.fillna()`

[Jake VanderPlas, 2017]

Handling Missing Values

```
import pandas as pd
```

```
user_data = {'first_name': ['Sammy', 'Jesse', np.nan, 'Jamie'],  
             'last_name': ['Shark', 'Octopus', np.nan, 'Mantis shrimp'],  
             'online': [True, np.nan, False, True],  
             'followers': [987, 432, 321, np.nan]}
```

```
df = pd.DataFrame(user_data, columns = ['first_name', 'last_name', 'online', 'followers'])
```

```
print(df)
```

	first_name	last_name	online	followers
0	Sammy	Shark	True	987.0
1	Jesse	Octopus	NaN	432.0
2	NaN	NaN	False	321.0
3	Jamie	Mantis shrimp	True	NaN

[Jake VanderPlas, 2017]

Handling Missing Values

```
df_drop_missing = df.dropna()  
  
print(df_drop_missing)
```

```
first_name last_name online  followers  
0      Sammy    Shark   True      987.0
```

[Jake VanderPlas, 2017]

Handling Missing Values

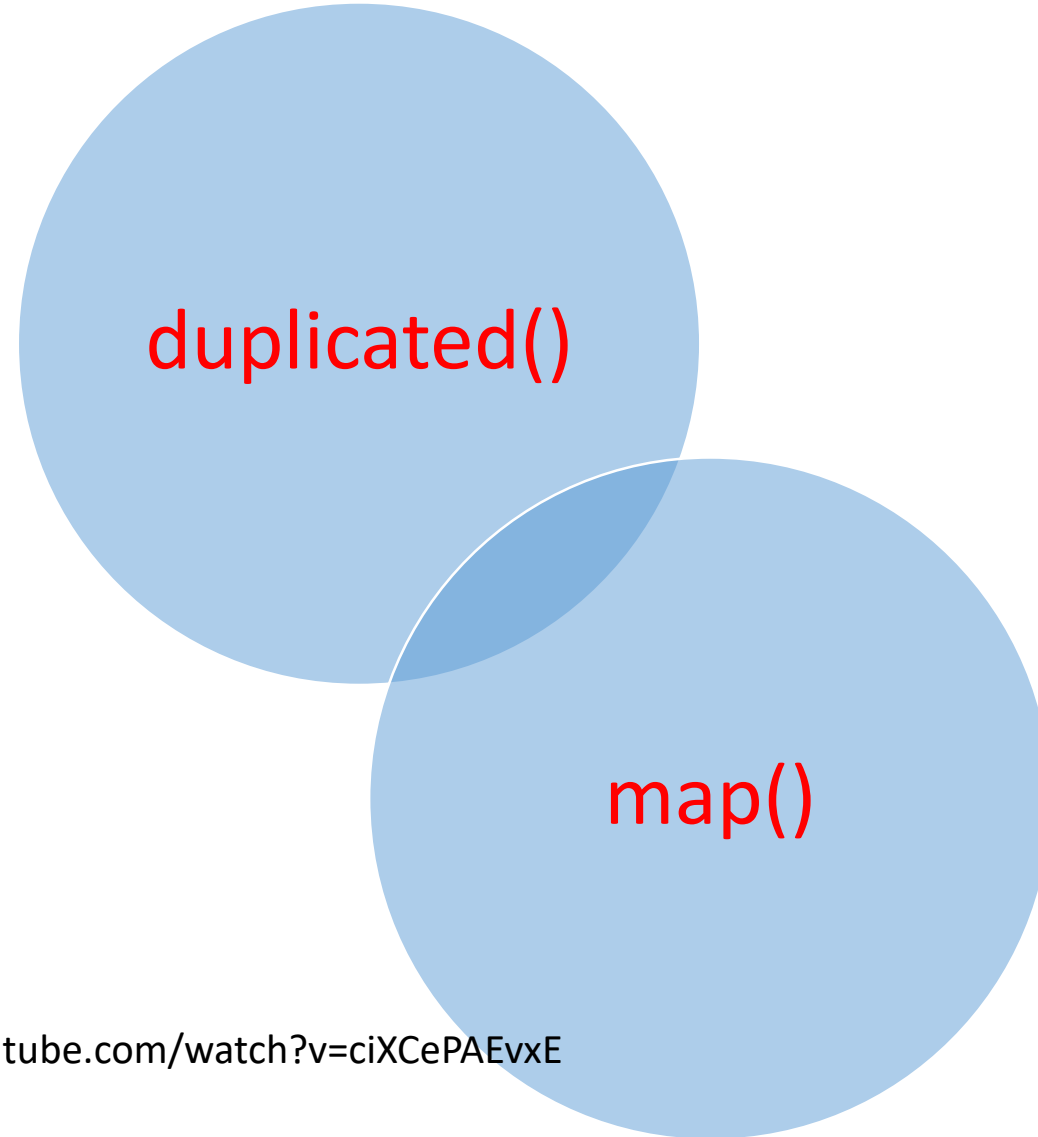
```
df_fill = df.fillna(0)

print(df_fill)
```

	first_name	last_name	online	followers
0	Sammy	Shark	True	987.0
1	Jesse	Octopus	0	432.0
2	0	0	False	321.0
3	Jamie	Mantis shrimp	True	0.0

[Jake VanderPlas, 2017]

Removing Duplicates



<https://www.youtube.com/watch?v=ciXCePAEvxE>

Example

Attr 1	Atrr2
Jonh	K
Ann	Na
Jonh	Len
Jonh	K

<https://www.youtube.com/watch?v=ciXCePAEvxE>

Removing Duplicates

Mapping

```
In [51]: data = pd.DataFrame({'food': ['Bacon', 'Pork', 'Bacon', 'Pastrami', 'Beef', 'Bacon', 'Pastrami', 'Ham', 'Lox'],  
                             'ounces': [4, 3, 12, 6, 7.5, 8, 3, 5, 6]})
```

```
In [52]: data
```

```
Out[52]:
```

	food	ounces
0	Bacon	4.0
1	Pork	3.0
2	Bacon	12.0
3	Pastrami	6.0
4	Beef	7.5
5	Bacon	8.0
6	Pastrami	3.0
7	Ham	5.0
8	Lox	6.0

```
In [53]: meat_to_animal = {'bacon': 'pig', 'pork': 'pig', 'pastrami': 'cow', 'beef': 'cow', 'ham': 'pig', 'lox': 'salmon'}
```

Removing Duplicates

Mapping

```
In [54]: lowercased = data['food'].str.lower()
```

```
In [55]: lowercased
```

```
Out[55]: 0      bacon
1      pork
2      bacon
3  pastrami
4      beef
5      bacon
6  pastrami
7      ham
8      lox
Name: food, dtype: object
```

<https://www.youtube.com/watch?v=ciXCePAEvxE>

Removing Duplicates

Mapping

```
In [56]: data['animal'] = lowercased.map(meat_to_animal)
```

```
In [57]: data
```

```
Out[57]:
```

	food	ounces	animal
0	Bacon	4.0	pig
1	Pork	3.0	pig
2	Bacon	12.0	pig
3	Pastrami	6.0	cow
4	Beef	7.5	cow
5	Bacon	8.0	pig
6	Pastrami	3.0	cow
7	Ham	5.0	pig
8	Lox	6.0	salmon

<https://www.youtube.com/watch?v=ciXCePAEvxE>

การเลือกคอลัมน์ใน pandas DataFrame

```
In [1]: import pandas as pd
```

```
In [2]: df = pd.read_csv('https://github.com/prasertcbs/tutorial/raw/master/score.csv', nrows=5)  
df
```

```
Out[2]:
```

	studentID	gender	math1	math2	math3	eng1	eng2	eng3	sci1	sci2	sci3
0	1	M	6	8	9	4	11	12	5	19	9
1	2	M	5	8	8	10	15	9	15	18	20
2	3	M	9	8	10	14	8	13	6	10	14
3	4	F	10	9	7	4	8	13	20	8	15
4	5	F	5	10	6	13	10	14	8	12	16

<https://github.com/prasertcbs/tutorial/raw/master/score.csv>

การเลือกคอลัมน์ใน pandas DataFrame

การเลือกคอลัมน์ eng1, eng2, eng3 ขึ้นมาแสดง

```
In [3]: df[['eng1', 'eng2', 'eng3']]
```

```
Out[3]:
```

	eng1	eng2	eng3
0	4	11	12
1	10	15	9
2	14	8	13
3	4	8	13
4	13	10	14

<https://www.youtube.com/watch?v=ciXCePAEvxE>

การเลือกคอลัมน์ใน pandas DataFrame

การเลือกคอลัมน์ eng1, eng2, eng3 ขึ้นมาแสดง สามารถใช้ “loc” มาใช้ในการระบุคอลัมน์ ที่ต้องการ

```
In [4]: df.loc[:, 'eng1':'eng3']
```

```
Out[4]:
```

	eng1	eng2	eng3
0	4	11	12
1	10	15	9
2	14	8	13
3	4	8	13
4	13	10	14

<https://www.youtube.com/watch?v=ciXCePAEvxE>

Data Integration

การอ่านและรวมไฟล์ CSV หลาย ๆ ไฟล์

Branch: master ▾

tutorial / eod /

Create new fileFind fileHistory

 prasertcbs End Of Day stock price

Latest commit 817d478 on Oct 17, 2017

..

 bbl.csv	End Of Day stock price	11 months ago
 ktb.csv	End Of Day stock price	11 months ago
 scb.csv	End Of Day stock price	11 months ago

<https://github.com/prasertcbs/tutorial/raw/master/eod/>

ตัวอย่าง

การอ่านและรวมไฟล์ CSV หลาย ๆ ไฟล์ ก่อนการนำไปประมวลผล

```
In [9]: stocks = ['bbl', 'ktb', 'scb']  
dfs = []  
location_prefix = 'https://github.com/prasertcbs/tutorial/raw/master/eod/'  
dfs = [pd.read_csv(f'{location_prefix}{s}.csv', parse_dates=True, index_col='Date') for s in stocks]
```

```
In [10]: type(dfs)
```

```
Out[10]: list
```

<https://www.youtube.com/watch?v=ciXCePAEvxE>

ตัวอย่าง

การอ่านและรวมไฟล์ CSV หลาย ๆ ไฟล์ ก่อนการนำไปประมวลผล

In [11]: `dfs[0]`

Out[11]:

	Open	High	Low	Close	Adj Close	Volume
Date						
2017-02-01	174.5	175.5	173.5	175.5	169.355362	5953500.0
2017-02-02	175.0	177.0	174.5	175.5	169.355362	5250500.0
2017-02-03	175.0	178.5	175.0	178.5	172.250336	5470600.0
2017-02-06	178.5	181.5	178.5	180.5	174.180313	8150800.0
2017-02-07	180.5	181.0	179.0	180.0	173.697815	3648600.0
2017-02-08	180.0	180.0	178.5	179.0	172.732819	2574200.0
2017-02-09	179.5	182.5	179.5	181.0	174.662796	7290700.0
2017-02-10	182.0	182.5	181.0	182.5	176.110275	3589100.0
2017-02-13	182.5	182.5	182.5	182.5	176.110275	0.0
2017-02-14	182.0	182.5	179.0	179.0	172.732819	2841700.0

<https://www.youtube.com/watch?v=ciXCePAEvxE>

ตัวอย่าง

การอ่านและรวมไฟล์ CSV หลาย ๆ ไฟล์ ก่อนการนำไปประมวลผล

In [12]: `dfs[2]`

Out[12]:

	Open	High	Low	Close	Adj Close	Volume
Date						
2017-02-01	150.5	153.0	150.5	152.0	146.727814	6969200.0
2017-02-02	152.0	153.0	151.0	151.0	145.762512	6529100.0
2017-02-03	151.0	153.5	151.0	153.0	147.693130	6597700.0
2017-02-06	154.0	155.0	153.5	154.5	149.141098	9901200.0
2017-02-07	154.5	155.0	153.5	154.0	148.658447	3802600.0
2017-02-08	153.5	154.5	153.0	154.0	148.658447	5747300.0
2017-02-09	154.5	156.0	153.5	154.0	148.658447	6157400.0
2017-02-10	154.5	155.0	154.0	154.0	148.658447	9620500.0
2017-02-13	154.0	154.0	154.0	154.0	148.658447	0.0
2017-02-14	155.0	156.0	153.0	154.0	148.658447	9754000.0

<https://www.youtube.com/watch?v=ciXCePAEvxE>

ตัวอย่าง

การอ่านและรวมไฟล์ CSV หลาย ๆ ไฟล์ ก่อนการนำไปประมวลผล

```
In [13]: df = pd.concat(dfs)
df
```

Out[13]:

	Open	High	Low	Close	Adj Close	Volume
Date						
2017-02-01	174.500000	175.500000	173.500000	175.500000	169.355362	5953500.0
2017-02-02	175.000000	177.000000	174.500000	175.500000	169.355362	5250500.0
2017-02-03	175.000000	178.500000	175.000000	178.500000	172.250336	5470600.0
2017-02-06	178.500000	181.500000	178.500000	180.500000	174.180313	8150800.0
2017-02-07	180.500000	181.000000	179.000000	180.000000	173.697815	3648600.0
2017-02-08	180.000000	180.000000	178.500000	179.000000	172.732819	2574200.0
2017-02-09	179.500000	182.500000	179.500000	181.000000	174.662796	7290700.0
2017-02-10	182.000000	182.500000	181.000000	182.500000	176.110275	3589100.0
2017-02-13	182.500000	182.500000	182.500000	182.500000	176.110275	0.0
2017-02-14	182.000000	182.500000	179.000000	179.000000	172.732819	2841700.0
2017-02-01	19.000000	19.200001	18.900000	19.200001	18.366062	21752200.0
2017-02-02	19.200001	19.200001	18.900000	19.000000	18.174747	40206000.0

<https://www.youtube.com/watch?v=ciXCePAEvxE>

Dates and Times in Python

ออบเจกต์พิเศษใน datetime

มอดูล datetime นั้นมีการนิยามคลาสของออบเจกต์สำหรับเก็บค่าวันเดือนปีและเวลาโดยเฉพาะ มีอยู่ ๔ ชนิดคือ

<code>datetime.date</code>	ออบเจกต์เก็บค่าวันเดือนปี
<code>datetime.time</code>	ออบเจกต์เก็บค่าเวลา
<code>datetime.datetime</code>	เป็นออบเจกต์ที่เอา <code>datetime.date</code> กับ <code>datetime.time</code> มารวมกัน เก็บค่าทั้งวันเดือนปีและเวลา
<code>datetime.timedelta</code>	ออบเจกต์เก็บค่าระยะห่างระหว่างเวลาซึ่งมีหน่วยเป็นวันและวินาที

<https://www.youtube.com/watch?v=ciXCePAEvxE>

Dates and Times in Python

```
In [68]: from datetime import datetime  
datetime(year=2019, month = 9, day=2)
```

```
Out[68]: datetime.datetime(2019, 9, 2, 0, 0)
```

<https://www.youtube.com/watch?v=ciXCePAEvxE>

Dates and Times in Python

```
In [69]: from dateutil import parser  
date = parser.parse("2nd of September, 2019")  
date
```

```
Out[69]: datetime.datetime(2019, 9, 2, 0, 0)
```

<https://www.youtube.com/watch?v=ciXCePAEvxE>

Typed arrays of times: NumPy's datetime64

```
In [10]: import numpy as np  
date = np.array('2019-09-02', dtype = np.datetime64)  
date
```

```
Out[10]: array('2019-09-02', dtype='datetime64[D]')
```

```
In [11]: date+np.arange(12)
```

```
Out[11]: array(['2019-09-02', '2019-09-03', '2019-09-04', '2019-09-05',  
               '2019-09-06', '2019-09-07', '2019-09-08', '2019-09-09',  
               '2019-09-10', '2019-09-11', '2019-09-12', '2019-09-13'],  
              dtype='datetime64[D]')
```

<https://www.youtube.com/watch?v=ciXCePAEvxE>

การแสดงวันและเวลาในรูปแบบต่าง ๆ

```
In[6]: np.datetime64('2015-07-04')
```

```
Out[6]: numpy.datetime64('2015-07-04')
```

```
In[7]: np.datetime64('2015-07-04 12:00')
```

```
Out[7]: numpy.datetime64('2015-07-04T12:00')
```

```
In[8]: np.datetime64('2015-07-04 12:59:59.50', 'ns')
```

```
Out[8]: numpy.datetime64('2015-07-04T12:59:59.500000000')
```

<https://www.youtube.com/watch?v=ciXCePAEvxE>

อ้างอิง

- Jesús Rogel-Salazar. DATA SCIENCE AND ANALYTICS WITH PYTHON CRC Press Taylor & Francis Group, 2017.
- Jake VanderPlas. Python Data Science Handbook . O'REILLY, 2017.